

```

package com.example.nuvem

import androidx.room.Entity
import androidx.room.PrimaryKey
import java.math.BigDecimal
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Locale

@Entity(tableName = "pedido")
data class Pedido(
    @PrimaryKey(autoGenerate = true) val numero: Int = 0,
    val cpf: String,

    // CORRIGIDO: Usa o Calendar para pegar a data corrente no formato AAAA-MM-DD
    val data_pedido: String = SimpleDateFormat("yyyy-MM-dd",
        Locale.getDefault()).format(Calendar.getInstance().time),

    val total_pedido: BigDecimal,
    val endereco_entrega: String,
    val bairro_entrega: String,
    val cep_entrega: String,
    val telefone_entrega: String
)

```

```

package com.example.nuvem

import androidx.room.Entity
import androidx.room.ForeignKey
import java.math.BigDecimal

@Entity(
    tableName = "itempedido",
    // Declarando a relação com a tabela pedido
    foreignKeys = [
        ForeignKey(
            entity = Pedido::class, // A classe da tabela Pai
            parentColumns = ["numero"], // O campo chave na tabela Pedido
            childColumns = ["pedido"] // O campo que faz o vínculo aqui na tabela ItemPedido
        )
    ], // <-- Essa vírgula aqui é obrigatória para separar os parâmetros
    primaryKeys = ["pedido", "produto"]
)
data class ItemPedido(
    val pedido: Int,
    val produto: String,
    val quantidade: Int,
    val preco_venda: BigDecimal,
    val total_item: BigDecimal
)

```

Eu tenho essas duas tabelas definidas e um cardView com todos os produtos, podendo ser filtrados. O que eu gostaria:

- 1) Selecionar um produto no cardView que chamaria uma activity com o produto escolhido e uma opção de colocar a quantidade.
- 2) Nessa nova activity terias as opções: Selecionar produto, Continuar Selecionando e Finalizar Pedido.
- 3) Ao finalizar o pedido abriria uma nova activity com todos os produtos selecionados e um spinner com as formas de pagamento oferecidas (vinda de uma tabela Room), o local de entrega que por default traria os dados já cadastrados em uma tabela chamada dados pessoais podendo ser editado,

caso os dados da entrega desse pedido seja diferente e as seguintes opções: enviar pedido, continuar comprando.

4) Quando o pedido fosse enviado, seria disparado um sms avisando sobre o novo pedido e iria atualizar o banco de dados que está hospedado na Aiven.